

# Java Practice Lab: Mastering Exception Handling

Dr. Hamza Djigal  
**Java Architect & Assistant Professor**  
Wenzhou-Kean University  
dhamza@kean.edu

November 12, 2025

**try • catch • finally • throw • throws • Custom  
Exceptions**

## Lab Overview

**Objective:** Strengthen your understanding of Java Exception Handling through small programs that demonstrate:

- Checked and unchecked exceptions
- try-catch-finally blocks
- throw and throws keywords
- Custom exception classes
- Exception propagation



## Exercise 1: Division by Zero Handling

**Goal:** Handle runtime errors caused by division by zero.

**Instructions:**

1. Create a new Java class `DivisionExample`.
2. Ask the user to enter two integers.
3. Perform the division inside a `try` block.
4. Catch any `ArithmeticException`.
5. Use `finally` to print “Execution completed.”

**Think about:**

- What happens if the denominator is zero?
  - Why is it important to close the scanner in `finally`?
- 

## Exercise 2: Array Index Exception

**Goal:** Prevent program crashes from invalid array index access.

**Instructions:**

1. Create an integer array with 5 elements.
2. Ask the user to enter an index between 0–4.
3. Use a `try{catch}` block to handle `ArrayIndexOutOfBoundsException`.

**Think about:**

- What happens if the user enters an index larger than 4?
- Why should we validate user input?



## Exercise 3: Multiple Exception Types

**Goal:** Handle different types of exceptions in a single program.

**Instructions:**

1. Accept two numbers from the user and divide them.
2. Store a value in an array index chosen by the user.
3. Use multiple `catch` blocks to handle:
  - `InputMismatchException`
  - `ArithmeticException`
  - `ArrayIndexOutOfBoundsException`

**Think about:**

- Which exception occurs first if multiple problems happen?
- What is the order of the `catch` blocks?

## Exercise 4: Custom Exception Class

**Goal:** Define your own exception to handle specific business rules.

**Instructions:**

1. Create a custom class `InsufficientBalanceException` extending `Exception`.
2. In `BankAccount`, throw this exception when a withdrawal exceeds the balance.
3. Catch and handle it in the main class `BankTest`.

**Think about:**

- Why do we need custom exceptions when Java has built-in ones?
- What's the difference between `throw` and `throws`?



## Exercise 5: Exception Propagation

---

**Goal:** Understand how exceptions move up the call stack.

**Instructions:**

1. Create three methods: `methodA()`, `methodB()`, and `methodC()`.
2. Let `methodC()` throw an exception.
3. Let `methodA()` handle it while `methodB()` passes it up.

**Think about:**

- What does it mean when we say an exception “propagates”?
- How is this related to using `throws` in method signatures?

## Concepts Demonstrated

- **try-catch-finally:** Structured exception handling.
- **Multiple catch:** Handling different exceptions separately.
- **throw and throws:** Creating and declaring exceptions.
- **Custom exceptions:** Defining domain-specific errors.
- **Propagation:** Passing exceptions through methods.